

Software Development Fundamentals (SDF) – I

ODD 2023



Arrays

(Lecture 15-21)

Jaypee Institute of Information Technology (JIIT)
A 10, Sector 62, Noida

SDF – I: Lecture Plan for Module 3 (6 Lectures)

Lecture No.	Topics to be Discussed
15	Introduction to Arrays (Declaration & Usage)
16	Input/Output using Arrays
17	Introduction to Strings (Declaration & Usage)
18	String library functions and its programs
19-20	Introduction to n-D Arrays (Declaration and Usage)
21	Input/Output using 2-D Arrays, 3-D Arrays

Why Arrays?

Suppose you want to store marks of five students, how will you do?

- First you have to see what will be the data type of marks?
- Assuming marks are stored as integers, we need five integers as:
 - `int marks1, marks2, marks3, marks4, marks5;`
- Why it is not acceptable?
 - Memorising five variables names for programmers
 - Accessing these variables from different memory location may create time overhead due to locality of reference.
 - Programmatically, we can't use for loop to access these variables.
 - To calculate average salary, total salary etc. is cumbersome.

Why arrays? (contd...)

- To solve the discussed problem, we can use array as:

```
int marks[5];
```

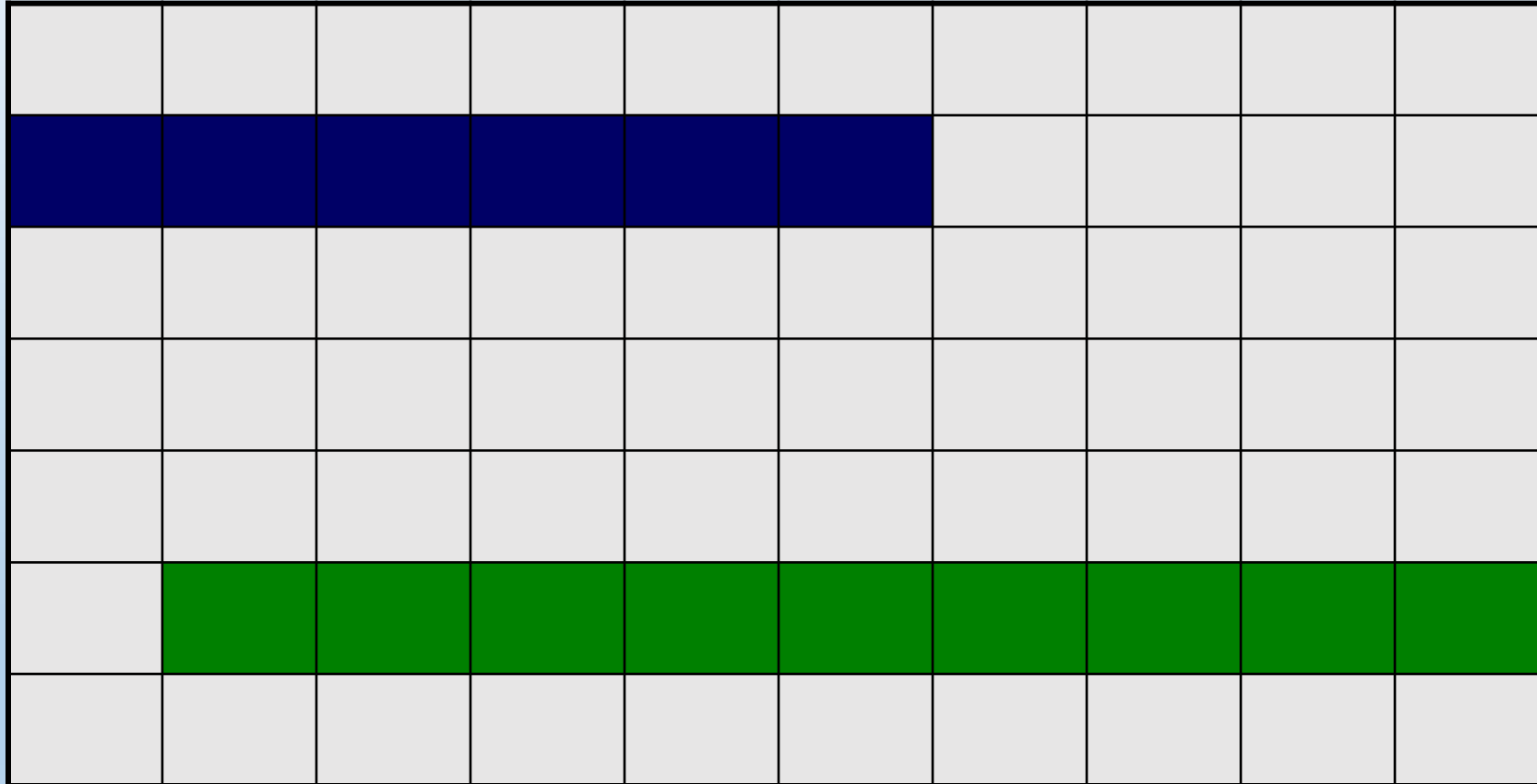
- It represents declaration of five integer elements together.
- We don't need to memorise five variable names; only one is sufficient.
- As these five integers are declared together, they are assigned contiguous memory location.
- Programmatically, we can use for loop to access these variables.

How to achieve these, we will see in next slides...

What is Array?

- Array is a composite data type to store homogeneous data types together.
- An array is assigned a contiguous block of memory for its elements.
- The size of array is sum of size of its elements.
- Array is linear and indexed data structure.

Array as a contiguous block in memory



How to declare an array?

- To declare an integer variable, the syntax is:
`int marks;`
- To declare 5 integer variables using array, the syntax is:
`int marks[5];`
- The general syntax of array declaration is:
`<data-type> var[size];`

Common errors while declaration

- Empty array size

`int marks[]; //error`

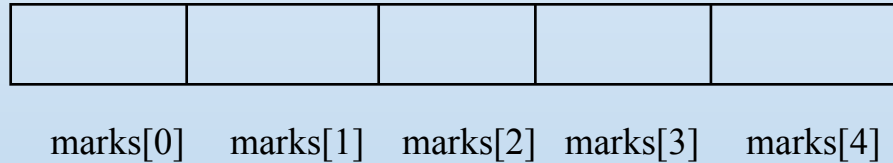
Note: the size of array can't be empty as compiler needs to know how much space to allocate.

- However, this is correct. Size of array is calculated from initialized elements.

`int marks[]={1, 4, 10};`

Memory layout of array declaration

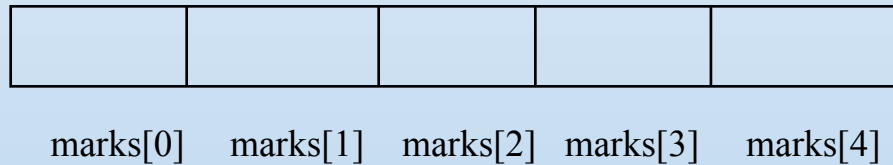
- The memory layout of array declaration can be represented as:
`int marks[5];`



- As can be seen, the indexes of array elements are from **0 to size-1**.
- The starting address of array is determined by the operating system.
- Therefore, the first element can be accessed as marks[0], second as marks[1] and so on.
- As array elements are not initialized, garbage values will be present.

Accessing array elements

- The array elements can be accessed using:
 <array_name>[index]



For example,

```
int marks[5];  
printf("%d",marks[1]);
```

// It will print value at index 1.

Assigning values to array elements

- Initialization while declaration

```
int marks[5]={5, 7, 9, 10, 8};  
print("%d", marks[4]); // it will print 8
```

- Using assignment operator

```
int marks[5];  
marks[0]=5;  
marks[1]=7;  
marks[2]=9;  
marks[3]=10;  
print("%d", marks[4]); // it will print garbage as marks[4] has not been initialized.
```

Out of bound checking

- In C, array's bound are not checked, programmers need to take care of that:

```
int marks[5]={5, 7, 9, 10, 8};
```

```
print ("%d", marks[5]); //it will print garbage value, no error.
```

sizeof operator in Arrays

- The sizeof operator with array returns the size of array which is calculated as:
 $\text{No. of elements} * \text{size of each element}$

For example,

```
int marks[5]={5, 7, 9, 10, 8};  
//it will print 20 assuming 64 bit compiler  
print ("%d", sizeof(marks));
```

- The sizeof operator can also be used to find the size of each element of the array as:

```
int marks[5]={5, 7, 9, 10, 8};  
//it will print 4 assuming 64 bit compiler  
print ("%d", sizeof(marks[0]));
```

Printing array values using for loop

- Array values can be printed using for loop as:

```
int marks[5]={5, 7, 9, 10, 8};
```

```
for(int i=0;i<5;i++)
```

```
{
```

```
printf("%d", marks[i]);
```

```
}
```

Assigning array values using for loop

- Array values can be assigned using for loop as:

```
int marks[5];  
for(int i=0;i<5;i++)  
{  
marks[i]=i+1;  
}
```

- It will assign 1 to marks[0], 2 to marks[1] and so on. Resultant array will be:
1, 2, 3, 4, 5

Assigning array values using scanf

- Array values can be assigned using for loop as:

```
int marks[5];  
// it will take inputs from users and assign to array elements  
for(int i=0;i<5;i++)  
{  
printf("Enter marks of student %d\n", i);  
scanf("%d", &marks[i]);  
}  
// it will print the values assigned to array elements  
for(int i=0;i<5;i++)  
{  
printf("The marks of student %d are\t", i);  
printf("%d\n", marks[i]);  
}
```

Finding whether an element exists in an array or not

```
#include <stdio.h>
#include<stdlib.h>
int main()
{
    int arr[50];
    int key, size;
    printf("Enter size of array");
    scanf("%d", &size);
    printf("Enter array elements");
    for(int i=0;i<size;i++)
        scanf("%d", &arr[i]);
    printf("Enter element to be searched");
    scanf("%d", &key);
    // This loop searches the key in the array
    for(int i=0;i<size;i++)
    {
        if(arr[i]==key)
        {
            printf("elements exists");
            exit(0);
        }
    }
    printf("element does not exist");
    return 0;
}
```

Finding minimum element from an array

```
#include<stdio.h>
int main()
{
int arr[50];
int min, size;
printf("Enter size of array");
scanf("%d", &size);
printf("Enter array elements");
for(int i=0;i<size;i++)
scanf("%d", &arr[i]);
min=arr[0];
    // This loop finds the minimum from an array
    for(int i=1;i<size;i++)
    {
        if(arr[i]<min)
            min=arr[i];
    }
printf("Minimum element is: %d", min);
return 0;
}
```

Finding count of an element in an array

```
include<stdio.h>
int main()
{
int arr[50];
int key, size, count;
printf("Enter size of array");
scanf("%d", &size);
printf("Enter array elements");
for(int i=0;i<size;i++)
scanf("%d", &arr[i]);
printf("Enter the element to be counted");
scanf("%d", &key);
count=0;
    // This loop finds count of element in array
    for(int i=0;i<size;i++)
    {
        if(arr[i]==key)
            count++;
    }
printf("The count of element %d is %d", key, count);
return 1;
}
```

Write a C program to insert element in array at specified position.

Example

Input

Input array elements: 10, 20, 30, 40, 50

Input element to insert: 25

Input position where to insert: 3

Output

Elements of array are: 10, 20, 25, 30, 40, 50

```

#include <stdio.h>
#define MAX_SIZE 100

int main()
{
    int arr[MAX_SIZE];
    int i, size, num, pos;

    /* Input size of the array */
    printf("Enter size of the array : ");
    scanf("%d", &size);

    /* Input elements in array */
    printf("Enter elements in array : ");
    for(i=0; i<size; i++)
    {
        scanf("%d", &arr[i]);
    }

    /* Input new element and position to insert */
    printf("Enter element to insert : ");
    scanf("%d", &num);
    printf("Enter the element position : ");
    scanf("%d", &pos);

```

```

/* If position of element is not valid */
if(pos > size+1 || pos <= 0)
{
    printf("Invalid position! Please enter position between 1 to %d",
size);
}
else
{
    /* Make room for new array element by shifting to right */
    for(i=size; i>=pos; i--)
    {
        arr[i] = arr[i-1];
    }

    /* Insert new element at given position and increment size */
    arr[pos-1] = num;
    size++;

    /* Print array after insert operation */
    printf("Array elements after insertion : ");
    for(i=0; i<size; i++)
    {
        printf("%d\t", arr[i]);
    }
}

return 0;
}

```

Write a C program to delete element from array at specified position.

Example

Input

Input array elements: 10 20 30 40 50

Input position to delete: 2

Output

Array elements: 10, 30, 40, 50

```

#include <stdio.h>
#define MAX_SIZE 100

int main()
{
    int arr[MAX_SIZE];
    int i, size, pos;

    /* Input size and element in array */
    printf("Enter size of the array : ");
    scanf("%d", &size);
    printf("Enter elements in array : ");
    for(i=0; i<size; i++)
    {
        scanf("%d", &arr[i]);
    }
    /* Input element position to delete */
    printf("Enter the element position to delete : ");
    scanf("%d", &pos);

```

```

/* Invalid delete position */
if(pos < 0 || pos > size)
{
    printf("Invalid position! Please enter position between 1
to %d", size);
}
else
{
    /* Copy next element value to current element */
    for(i=pos-1; i<size-1; i++)
    {
        arr[i] = arr[i + 1];
    }

    /* Decrement array size by 1 */
    size--;

    /* Print array after deletion */
    printf("\nElements of array after delete are : ");
    for(i=0; i<size; i++)
    {
        printf("%d\t", arr[i]);
    }
}

return 0;
}

```

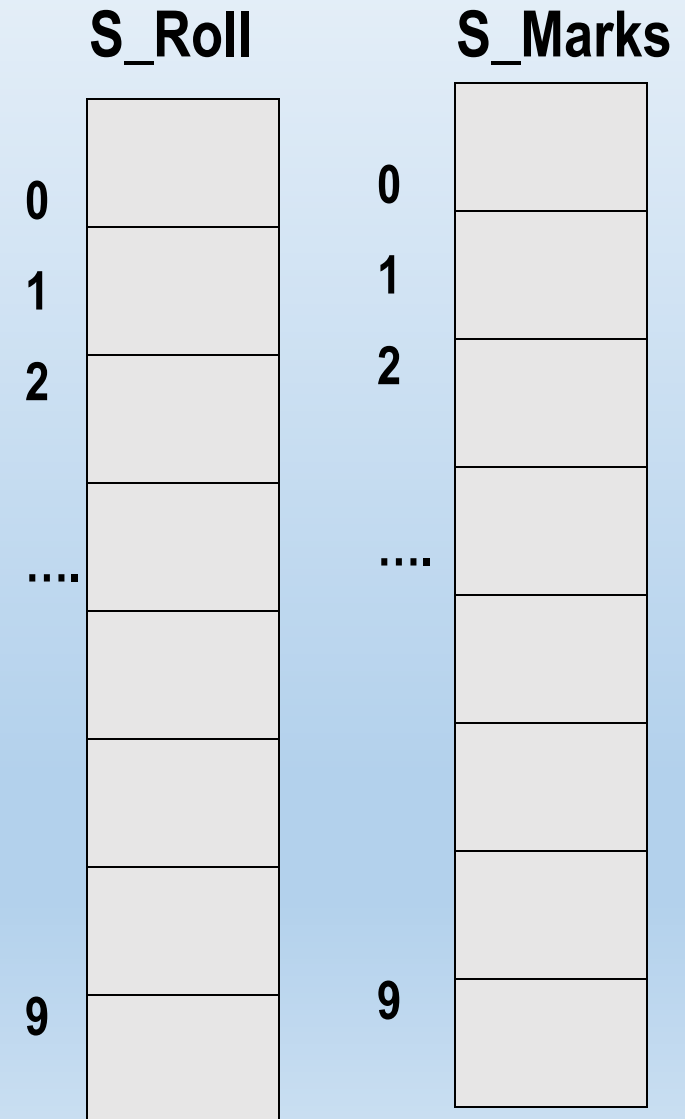
Exercise

In a university 10 students are there. WAP to:

- store their roll number and marks.
- Given roll number of a student, find his marks.

Solution

- Declare two arrays `int S_Roll[10]` and `int S_Marks [10]`
- For each student, input `S_Roll` and corresponding `S_Marks`
- Input `S_Roll_No` for which marks to be found.
- Search `S_Roll_No` in `S_Roll[ ]` and retrieve the index `i` at which `S_Roll_No` is found.
- From `S_Marks[ ]` array, display the value at index `i`



Program

```
#include<stdio.h>
#include<stdlib.h>
int main() {
    int S_Roll_No, S_Roll[10], S_Marks[10];
    printf("Enter roll no. and marks");

    // Input roll_no and marks
    for(int i=0;i<10;i++)
    {
        scanf("%d",&S_Roll[i]);
        scanf("%d",&S_Marks[i]);
    }
    printf("Enter roll no of student for which marks to be found");
    scanf("%d", &S_Roll_No);
    // Searching of marks for a given roll no
    for(int i=0;i<10;i++)
    {
        if(S_Roll[i]==S_Roll_No)
        {
            printf("%d",S_Marks[i]);
            exit(0);
        }
    }
    printf("Roll No. not found");
    return 0;
}
```

Character Array/ Strings

For grouping of characters such as storing name, address etc., character array/string is used.

For example,

```
char a ='c';  
// variable a of type character stores character 'c'.
```

```
char a[5]={'a', 'b', 'c', 'd', 'e'};  
// variable a is of type character array which stores 'a' at index 0, 'b' at index 1 and so on.
```

ASCII values of characters

Character	ASCII codes
A-Z	65-90
a-z	97-122
0-9	48-57
special characters	0-47, 58-64, 91-96, 123-127

NULL character

- It represents the end of string.
- scanf and gets function appends NULL character at the end of string.
- It is represented as `'\0'` and has ASCII value as 0.
- While printing the string, NULL character is used to find the **end of string**.

Character array initialization

`char b[10] = "JIIT";` → **NULL character is appended.**

`char c[10];
scanf("%s", c);` → **& is not required, %s is for storing all characters in a string, NULL character is appended**

`char d[5] = {'a', 'b', 'c', 'd', 'e'};` → **NULL character is not appended as individual elements of array are initialized.**

Printing values in character array

```
/* program to read user's name and display it */
```

```
int main( )
```

```
{
```

```
char name [10 ] ;
```

```
printf("Enter your name ");
```

```
scanf("%s ", name);
```

```
printf("Name is : %s", name);
```

→ **string stored in name is printed**

```
printf("Character at index 0 is %c", name[0]);
```

→ **character at 0th index is printed**

```
printf("ASCII value of character at index 0 is %d", name[0]);
```

→ **ASCII value of character at 0th index is printed**

```
return 0;
```

```
}
```

Limitation of scanning string using scanf

It assigns the string to the array till space is reached.

For example,

"Hello World" passed as string to scanf will store only "hello".

gets()

gets(s) : It collects a string of characters **terminated by new line character '\n'** from the input, appends NULL character '\0' and stores in s.

Unlike scanf, it allows input like spaces and tabs.

For example

```
char str [ 20 ] ;  
printf("enter string");  
// user enters "hello world" which is stored in str  
gets( str ) ;
```

puts()

puts(s): It prints the string s until NULL character is found and appends a new line character at the end.

For example,

```
char s[20] = "hello world";
```

```
puts (s) ; // it will print hello world
```

To count number of characters in a string

```
int main()
{
char s[20];
int len = 0;
printf("Enter string");
gets(s);
    //count characters until null character is reached
    while(s[len]!='\0')
        len++;
printf("%d", len);
}
```

To copy a string to another string

```
int main()
{
char s[20], t[20];
printf("Enter string");
gets(s);
    //copy s into t
    int i = 0;
    while(s[i]!='\0')
    {
        t[i]=s[i];
        i++;
    }
    // append '\0' at the end of t
    t[i]='\0';
puts(t);
}
```

To concatenate a string at the end of other string

```
int main()
{
char s[20], t[40];
printf("Enter string1");
gets(s);
printf("Enter string2");
gets(t);

//iterate till end of string s
int i = 0;
while(s[i]!='\0')
{
i++;
}

//Append t at the end of s
int j = 0;
while(t[j]!='\0')
{
s[i]=t[j];
i++;
j++;
}

// append '\0' at the end of s
s[i]='\0';
puts(s);
}
```

To reverse a string

```
int main()
{
char s[20];
printf("Enter string1");
gets(s);

//find len of string
int len = 0;
while(s[len]!='\0')
{
len++;
}

//swapping the characters in string to find the reverse
int i = 0, temp;
while(i<len/2)
{
temp=s[i];
s[i]=s[len-i-1];
s[len-i-1]=temp;
i++;
}
puts(s);
}
```

String Library Functions (<string.h> library)

Function	Description
strlen(s)	Finds the length of string s
strcpy(t,s)	Copies the string s to string t
strcat(s,t)	Concatenates t to the end of s
strcmp(s,t)	Compares two strings. It returns 0 if they are equal

strlen(), strcpy(), strcat() examples

```
int main()
{
char str1[20]="JIIT", str2[20];

printf("%d\n",strlen(str1)); //It will print 4

strcpy(str2,str1); //it will copy str1 into str2
puts(str2); // it will print "JIIT"

strcat(str1,str2); // it will append str2 at the end of str1
puts(str1); // it will print "JIITJIIT"

return 0;
}
```

strcmp() example

```
int main()
{
    char str1[20]="JIIT", str2[20]="JIIT", str3[20]="Noida";

    // prints 0 as str1 and str2 are equal
    printf("%d\n", strcmp(str1, str2));

    //prints difference between ascii value of first unmatched character
    printf("%d", strcmp(str1, str3));

    return 0;
}
```

Count number of letters and digits in a string

```
int main()
{
    char str1[50];
    gets(str1);
    int i =0, l=0, d=0;
    while(str1[i]!='\0')
    {
        if((str1[i]>='A' && str1[i]<='Z') || (str1[i]>='a' && str1[i]<='z'))
            l++;
        else if((str1[i]>='0' && str1[i]<='9'))
            d++;
        i++;
    }
    printf("The number of letters and digits in %s are: %d\t%d", str1, l, d);
    return 0;
}
```

Exercise:

- Write a C program to find highest frequency character in a string
- Write a C program to remove all repeated characters from a given string

Multidimensional Array

Suppose we want to store marks of 100 students in 5 subjects.

- Using 1-D array we need to create 5 1-D arrays of size 100 each.

Lets make situation more complex.

Suppose we want to store marks of 100 students in 5 subjects in 8 semester.

- 40 1-D arrays are required. 5 for 1st sem subjects, 5 for 2nd sem sub and so on.

Multidimensional array makes it simpler to store and access data of this type.

2-D Array Declaration

```
int m[3][4];
```

m[0]	m[0][0]	m[0][1]	m[0][2]	m[0][3]
m[1]	m[1][0]	m[1][1]	m[1][2]	m[1][3]
m[2]	m[2][0]	m[2][1]	m[2][2]	m[2][3]

- m is an 2-D array with three rows and four columns.
- It consists of three 1-D arrays each having size of four.
- 2-D array are stored as 1-D array in **row-major order**. It means that m[0] is stored first, then m[1] and m[2].

2-D Array Initialization

```
int m[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
```

or

```
int m[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

or

```
int m[][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

- In the first case, we have separated elements of 1st, 2nd and 3rd 1-D array by used nested braces.
- In the second case, first four elements will be assigned to 1st 1-D array. Similarly, next four elements will be assigned to 2nd 1-D array and so on.
- In the third case, number of rows are not declared, however, it will be automatically computed from the initialization.

Note: We can't leave number of column as blank unlike number of rows.

Referencing 2-D array elements

We need to specify two indexes to access elements in 2-D array.

```
int m[3][4];
```

```
m[0][1]=5; //5 will be stored in array at 0th row and 1st column
```

```
m[0][3]=6;
```

```
printf("%d", m[0][3]); // 6 will be printed
```

Storing and printing user data using 2-D array

```
int marks[3][4];
```

- marks is a 2-D array which can be used to store marks of different students for different subjects.
- Assuming row is representing student and column is representing marks.
marks[1][2] represents marks of 1st student in 2rd subject.

(subject no.)		→			
		[0] [1] [2] [3]			
(student no.)	marks [0]	40	62	60	61
	marks [1]	62	50	63	27
	marks [2]	55	75	59	87

Storing and accessing marks of students in different subjects

```
int main()
{
int marks[10][10];
int count_stud, count_sub, i, j;
printf("Enter number of students and subjects");
scanf("%d %d", &count_stud, &count_sub);
    // for each student representing a row, we are storing marks
    // of different subjects in different columns

    for(i=0;i<count_stud;i++)
    {
        printf("Enter marks of student %d\n", i+1);
        for(j=0;j<count_sub;j++)
        {
            printf("Enter marks of subject %d\n", j+1);
            scanf("%d", &marks[i][j]);
        }
    }
```

Storing marks of students in different subjects (contd...)

```
//printing marks of students in different subjects
for(i=0;i<count_stud;i++)
{
printf("The marks of students %d are:\n", i+1);
for(j=0;j<count_sub;j++)
{
printf("Subject 1: %d \n", marks[i][j]);
}
}
return 0;
}
```

Finding total marks of each student

		(subject no.)				
(student no.)			[0]	[1]	[2]	[3]
	marks	[0]	40	62	60	61
	marks	[1]	62	50	63	27
	marks	[2]	55	75	59	87

sum[]	
2	
2	0
3	1
0	
2	2
7	
6	

Solution:

- For each row, we have to find sum of all columns of that row.
- The number of elements in the sum[] array is equal to number of students.

Finding total marks of each student (contd...)

```
int sum[count_stud];  
    // calculating total marks for each student  
    for(i=0;i<count_stud;i++)  
    {  
        sum[i]=0;  
        for(j=0;j<count_sub;j++)  
        {  
            sum[i]+=marks[i][j];  
        }  
    }  
  
    // printing total marks for each student  
    for(i=0;i<count_stud;i++)  
    {  
        printf("Marks of student %d is %d\n", i, sum[i]);  
    }
```

C program to store temperature of two cities of a week and display it.

Output:

```
City 1, Day 1: 33
City 1, Day 2: 34
City 1, Day 3: 35
City 1, Day 4: 33
City 1, Day 5: 32
City 1, Day 6: 31
City 1, Day 7: 30
City 2, Day 1: 23
City 2, Day 2: 22
City 2, Day 3: 21
City 2, Day 4: 24
City 2, Day 5: 22
City 2, Day 6: 25
City 2, Day 7: 26
```

Displaying values:

```
City 1, Day 1 = 33
City 1, Day 2 = 34
City 1, Day 3 = 35
City 1, Day 4 = 33
City 1, Day 5 = 32
City 1, Day 6 = 31
City 1, Day 7 = 30
City 2, Day 1 = 23
City 2, Day 2 = 22
City 2, Day 3 = 21
City 2, Day 4 = 24
City 2, Day 5 = 22
City 2, Day 6 = 25
City 2, Day 7 = 26
```

```
#include<stdio.h>
const int CITY=2;
const int WEEK=7;

int main()
{
    int temperature[CITY][WEEK];
    int i ,j;

    /*Take input from user*/
    for(i=0;i<CITY;i++)
    {
        for(j=0;j<WEEK;j++)
        {
            printf("City[%d], Day[%d]: ", i+1, j+1);
            scanf("%d", &temperature[i][j] );
        }
        printf("\n");
    }

    /*Display output*/
    printf("Displaying Values:\n\n");

    for(i=0;i<CITY;i++)
    {
        for(j=0;j<WEEK;j++)
        {
            printf("City[%d], Day[%d]=%d\n", i+1, j+1, temperature[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Matrix addition using 2-D array

```
int main()
{
int mat1[10][10], mat2[10][10], mat3[10][10];
int n_rows, n_cols, i, j;
printf("enter number of rows and columns");
scanf("%d %d", &n_rows, &n_cols);

//input matrix 1
printf("enter elements of matrix 1");
for(i=0;i<n_rows;i++)
    for(j=0;j<n_cols;j++)
        scanf("%d", &mat1[i][j]);

//input matrix 2
printf("enter elements of matrix 2");
for(i=0;i<n_rows;i++)
    for(j=0;j<n_cols;j++)
        scanf("%d", &mat2[i][j]);
```

Matrix addition using 2-D array (contd...)

```
//adding mat1 and mat2
for(i=0;i<n_rows;i++)
    for(j=0;j<n_cols;j++)
        mat3[i][j]=mat1[i][j]+mat2[i][j];

//printing mat3
printf("The sum of mat1 and mat2 is:\n");
for(i=0;i<n_rows;i++)
{
    printf("\n");
    for(j=0;j<n_cols;j++)
        printf("%d\t", mat3[i][j]);
}

return 0;
}
```

Matrix transpose using 2-D array

```
int main()
{
    int mat1[10][10], mat2[10][10];
    int n_rows, n_cols, i, j;
    printf("enter number of rows and columns");
    scanf("%d %d", &n_rows, &n_cols);
    //input mat1
    printf("enter elements of matrix");
    for(i=0;i<n_rows;i++)
        for(j=0;j<n_cols;j++)
            scanf("%d", &mat1[i][j]);

    //finding transpose of mat1
    for(i=0;i<n_rows;i++)
        for(j=0;j<n_cols;j++)
            mat2[j][i] = mat1[i][j];
```

```
//printing transposed matrix
printf("The transposed matrix is: \n");
for(i=0;i<n_cols;i++)
{
    printf("\n");
    for(j=0;j<n_rows;j++)
        printf("%d\t", mat2[i][j]);
}
return 0;
}
```

Write a C program to read elements in a matrix and find the sum of elements of each row and columns of matrix.

```
Enter elements in matrix of size 3x3:
```

```
1 2 3
```

```
4 5 6
```

```
7 8 9
```

```
Sum of elements of Row 1 = 6
```

```
Sum of elements of Row 2 = 15
```

```
Sum of elements of Row 3 = 24
```

```
Sum of elements of Column 1 = 12
```

```
Sum of elements of Column 2 = 15
```

```
Sum of elements of Column 3 = 18
```

```
#include <stdio.h>

#define SIZE 3 // Matrix size

int main()
{
    int A[SIZE][SIZE];
    int row, col, sum = 0;

    /* Input elements in matrix from user */
    printf("Enter elements in matrix of size %dx%d: \n", SIZE,
SIZE);
    for(row=0; row<SIZE; row++)
    {
        for(col=0; col<SIZE; col++)
        {
            scanf("%d", &A[row][col]);
        }
    }
}
```

```
/* Calculate sum of elements of each row of matrix */
for(row=0; row<SIZE; row++)
{
    sum = 0;
    for(col=0; col<SIZE; col++)
    {sum += A[row][col]; }
    printf("Sum of elements of Row %d = %d\n", row+1, sum); }

/* Find sum of elements of each columns of matrix */
for(row=0; row<SIZE; row++)
{ sum = 0;
    for(col=0; col<SIZE; col++)
    {
        sum += A[col][row];
    }
    printf("Sum of elements of Column %d = %d\n", row+1,
sum);
}return 0;}
```

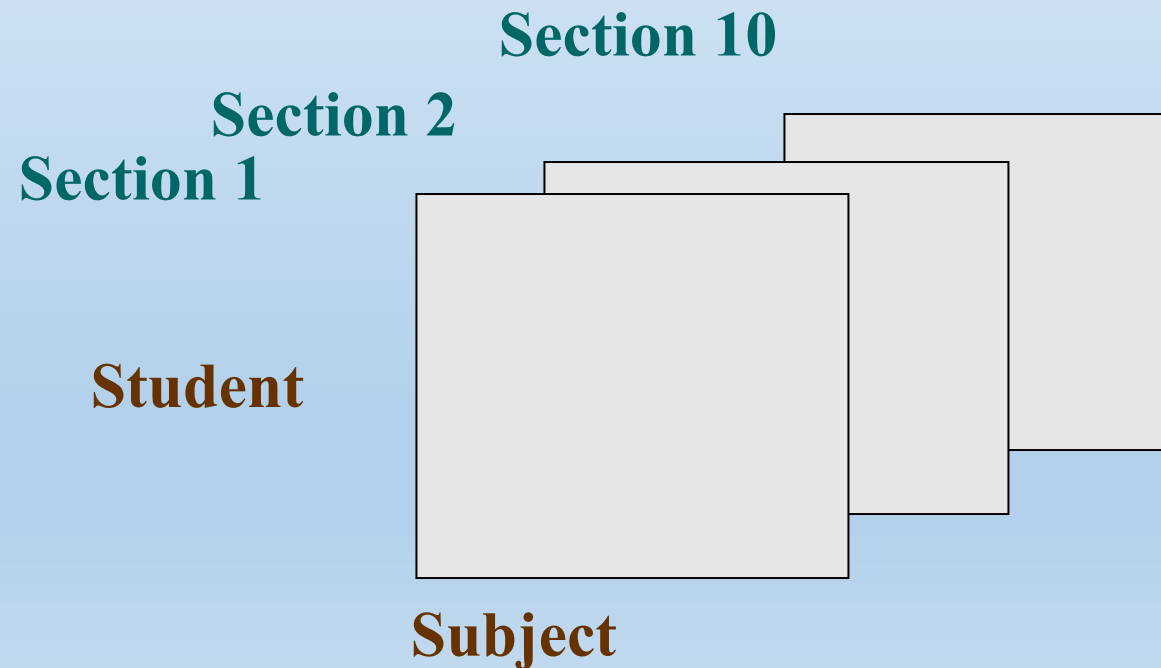
3-D Array

Suppose we want to store marks of 10 sections where each section have 50 students. For each student we want to store marks of 5 subjects.

3-D array declaration

```
int marks[10][50][5];
```

It can be visualised as 10 matrices each of size 50*5.



Storing and accessing marks of different sections' students in different subjects

```
int main()
{
    int marks[10][10][10];
    int count_sec, count_stud, count_sub, i, j, k;
    printf("Enter number of sections, students and subjects");
    scanf("%d %d %d", &count_sec, &count_stud, &count_sub);
    /*for each section, there exist a 2-D array in which student represents a row and subject represents a column*/
    for(i=0;i<count_sec;i++)
    {
        for(j=0;j<count_stud;j++)
        {
            for(k=0;k<count_sub;k++)
            {
                printf("Enter marks of section %d, student %d and subject %d\n", i+1, j+1, k+1);
                scanf("%d", &marks[i][j][k]);
            }
        }
    }
}
```

Storing and accessing marks of different sections' students in different subjects

//printing marks of students of different sections in different subjects

```
for(i=0;i<count_sec;i++)
{
    for(j=0;j<count_stud;j++)
    {
        for(k=0;k<count_sub;k++)
        {
            printf("The marks of section %d, student %d and subject %d are %d \n", i+1, j+1, k+1, marks[i][j][k]);
        }
    }
}
return 0;
}
```

Highest marks in each subject

Solution:

The count of subjects is count_sub which is size of output array.

For each subject, scan marks of students in each section to find the highest.

Code:

```
int max[count_sub];
//for each subject, scan each section's students marks and find the highest
for(k=0;k<count_sub;k++)
{
    max[k]=0;
    for(i=0;i<count_sec;i++)
    {
        for(j=0;j<count_stud;j++)
        {
            if(max[k]<marks[i][j][k])
            {
                max[k]=marks[i][j][k];
            }
        }
    }
}

printf("The maximum marks are\n");
for(i=0;i<count_sub;i++)
printf("Subject %d \t %d\n", i+1, max[i]);
```

Limitations of Array

- Static memory allocation
- Unoccupied space
 - declaration of 100 elements but only 10 are used.
- Insertion and deletion may be computationally expensive.
 - deletion of first element may require to shift remaining elements.
- Only homogeneous data types can be stored.

The End